杂题选讲

bdzzj & Diu

2026-11-02

Outline

1. Group 1
   1.1. qoj22881 Subset Sums
   1.2. qoj18316 插松枝
   1.3. qoj18428 Sort
   1.4. qoj14742 海鸥奇遇
   1.5. qoj18339 One, Two, Three
2. Group 2
   2.1. qoj18302 Remix
   2.2. qoj23137 Graph Built by XOR
   2.3. qoj23130 Orientation of a Graph
   2.4. qoj22886 Merge Sequence
   2.5. qoj7782 Ursa Minor

qoj22881 Subset Sums

statement

Source: 4th Ucup semifinal E

给定 n 个整数 a_i，将这 n 个整数的所有 2^n 个子集分别求和，将 2^n 个子集和记为 $s_1, s_2, s_3, \ldots, s_{2^n}$，然后将 s 数组按升序排序，求：

$$\max_{1 \le i < 2^n} (s_{i+1} - s_i)$$

$n \le 10^6, 0 \le a_i \le 10^9$。时限 $1s$。

qoj22881 Subset Sums

statement

Source: 4th Ucup semifinal E

给定 n 个整数 a_i，将这 n 个整数的所有 2^n 个子集分别求和，将 2^n 个子集和记为 $s_1, s_2, s_3, \ldots, s_{2^n}$，然后将 s 数组按升序排序，求：

$$\max_{1 \le i < 2^n} (s_{i+1} - s_i)$$

$n \le 10^6, 0 \le a_i \le 10^9$。时限 $1s$。

hint

把 a_i 从小到大排序之后增量考虑。

qoj22881 Subset Sums

- 首先将 a_i 按从小到大排序，然后从 a_1 到 a_n 考虑每次加入一个数会让答案发生什么变化。
- 记 $sum_i = \sum_{j=1}^{i} a_j$，记 $s_{i,1}, s_{i,2}, \dots s_{i,2^i}$ 表示前 i 个数的 2^i 个子集的子集和排完序之后的数组，记 ans_i 表示前 i 个数的答案，即 $ans_i = \max_{1 \le j < 2^i}(s_{i,j+1} - s_{i,j})$。
- 现在加入 a_{i+1} 这个数，那么把 $s_{i,1}, s_{i,2}, \dots s_{i,2^i}$ 和 $s_{i,1} + a_{i+1}, s_{i,2} + a_{i+1}, \dots s_{i,2^i} + a_{i+1}$ 这两个数组合并之后再排序就可以得到 $s_{i+1,1}, s_{i+1,2}, \dots, s_{i+1,2^{i+1}}$。
- 考虑什么情况下 $ans_{i+1} > ans_i$，对于 $s_{i+1,1}, s_{i+1,2}, \dots, s_{i+1,2^{i+1}}$ 这个数组中的相邻两个数 $s_{i+1,j}, s_{i+1,j+1}$，如果 $s_{i+1,j}$ 在合并前的数组中不是最后一个数或者 $s_{i+1,j+1}$ 在合并前的数组中不是第一个数，那么该数在合并前的数组中与其前驱/后继的差必定大于或等于 $s_{i+1,j+1} - s_{i+1,j}$，则 $s_{i+1,j+1} - s_{i+1,j} \le ans_i$，也就是说只有 $s_{i,1} + a_{i+1} - s_{i,2^i}$ 有可能大于 ans_i。

qoj22881 Subset Sums

- 而 $s_{i,1} + a_{i+1} - s_{i,2^i} = a_{i+1} - sum_i$，也就是当且仅当 $a_{i+1} - sum_i > ans_i$ 的情况下，$ans_{i+1} > ans_i$，并且此时 ans_{i+1} 就等于 $a_{i+1} - sum_i$。
- 那有没有可能 $ans_{i+1} < ans_i$ 呢，显然不可能，因为当某个 $j < i$ 满足 $a_{j+1} - sum_j > 0$ 时，由于 a 序列是单调不降的，所以后面继续加入数时 a_{j+1} 和 sum_j 这两个数在子集和数组中会一直相邻，所以有 $\forall i > j, ans_i \geq a_{j+1} - sum_j$，也就是说每次 ans 只会变大不会变小。
- 最后的结论就是：$ans_n = \max_{i=1}^{n}(a_i - sum_{i-1})$。
- 时间复杂度 $O(n \log n)$。

qoj18316 插松枝

statement

Source: SDCPC 2026 J

你有三个栈，初始二号栈和三号栈都是空的，一号栈有 n 个元素，从栈顶到栈底依次为 a_i。

你可以进行三种操作：

1. 将一号栈栈顶的元素放到二号栈的栈顶。
2. 将二号栈栈顶的元素放到一号栈的栈顶。
3. 将一号栈栈顶的元素放到三号栈的栈顶，同时需要保证操作完之后三号栈中的元素从栈顶到栈底是单调不降的。

你的目标是将所有元素移动到三号栈，求最小操作次数，不需要构造方案。

$n \le 4 \times 10^5, a_i \le 10^9$。时限 $2s$。

qoj18316 插松枝

- 首先一号栈和二号栈的机制相当于有一个“光标”可以在 a_i 序列中移动，每次可以移动一格，或者将光标右边的元素删除，而三号栈单调不降则代表你需要将序列中的元素从大到小依次删除。
- 如果 a_i 中所有元素互不相同，那直接模拟即可，因为每一步需要删除的数是固定的，直接将光标移动过去删除即可。
- 考虑 a_i 中的一个值 p，记这个值在序列中最左边的位置为 L_p，最右边的位置为 R_p。现在要将序列中所有 p 都删除，那么显然一定是先将光标移动到 L_p 再移动到 R_p，或是先移动到 R_p 再移动到 L_p，并在移动的过程中将所有 p 都删除了。
- 注意到删除完所有 p 之后光标的位置一定在 L_p 或是 R_p，因此可以设计 $dp_{p,0/1}$ 表示删除完了所有 $\geq p$ 的值之后光标在 L_p/R_p 的位置所需要的最小操作次数。转移也很简单，离散化之后 $dp_{p,0/1}$ 直接从 $dp_{p+1,0/1}$ 转移过来，移动光标的操作次数直接用树状数组计算即可。
- 时间复杂度 $O(n \log n)$。

qoj18428 Sort

statement

Source: HBCPC 2026 A

给定一个长度为 n 的排列 p。

每次操作从 $[1,n]$ 中随机一个整数 i，然后将排列 p 的区间 $[1,i]$ 和 $[i+1,n]$ 分别按升序排序，特别地，当 $i=n$ 时，直接将整个排列排序。

求期望多少次操作后整个排列变成升序（即 $p_i=i$），答案对 998244353 取模。

多测，$n\le 500,\sum n\le 2000$。时限 $4s$。

qoj18428 Sort

statement

Source: HBCPC 2026 A
给定一个长度为 n 的排列 p。
每次操作从 $[1,n]$ 中随机一个整数 i，然后将排列 p 的区间 $[1,i]$ 和 $[i+1,n]$ 分别按升序排序，特别地，当 $i=n$ 时，直接将整个排列排序。
求期望多少次操作后整个排列变成升序（即 $p_i=i$），答案对 998244353 取模。
多测，$n\le 500,\sum n\le 2000$。时限 $4s$。

hint

考虑第一次操作之后序列会变成什么样子。

qoj18428 Sort

- 直接枚举第一次操作随机到的 k，那么此时序列就会被分成 $[1,k]$ 和 $[k+1,n]$ 两个递增序列，记这次操作完之后的序列为 p'。
- 考虑接下来的操作会让序列发生什么变化，假设接下来的操作随机到 i，由于左右对称不妨设 $i \in [1,k]$，如果 $p'_i < p'_{k+1}$，那就直接排完序了，否则就相当于把 $[i+1,k]$ 这部分合并到右边 $[k+1,n]$。继续考虑后续的操作会发现实际上就是一个不停将左边的一个后缀合并到右边，将右边的一个前缀合并到左边，直到左边的最大值小于右边的最小值就排序结束了。

qoj18428 Sort

- 因此我们直接设 $dp_{l,r}(l \in [1,k], r \in [k+1,n])$ 表示当前已经把 $[l+1,k]$ 合并到了右边，把 $[k+1,r-1]$ 合并到了左边，期望还需要操作多少次可以排好序。转移就是枚举下一次操作随机到了什么，如果随机到已经合并的部分那就直接结束了，否则就合并一部分到另一边，即：

$$dp_{l,r} = 1 + \frac{\sum_{i=1}^{l-1} dp_{i,r} + \sum_{i=r+1}^{n} dp_{l,i} + dp_{l,r}}{n}$$

- 解个方程得到转移式，然后直接前缀和优化即可 $O(n^2)$ 求一次 dp。
- 总时间复杂度 $O(n^3)$。

qoj14742 海鸥奇遇

statement

Source: LNCPC 2025 A

在一个 5000×5000 的网格中，初始有 n 只海鸥和 n 份食物，一个格子上只有一只海鸥/一份食物/什么都没有。

每次操作你可以选择一只海鸥和一份食物，要求这份食物刚好在这只海鸥的正上/下/左/右方，并且这份食物和这只海鸥之间的所有格子都是海鸥（即不能是空格子/另一份食物）。选择完之后就删除这只海鸥和这份食物，即这两个格子变成空格子。

你需要操作 n 次把所有海鸥和食物都删除完，或判定无解。需要输出方案。

多测，$\sum n \leq 5000$。时限 $1s$。

qoj14742 海鸥奇遇

statement

Source: LNCPC 2025 A

在一个 5000×5000 的网格中，初始有 n 只海鸥和 n 份食物，一个格子上只有一只海鸥/一份食物/什么都没有。

每次操作你可以选择一只海鸥和一份食物，要求这份食物刚好在这只海鸥的正上/下/左/右方，并且这份食物和这只海鸥之间的所有格子都是海鸥（即不能是空格子/另一份食物）。选择完之后就删除这只海鸥和这份食物，即这两个格子变成空格子。

你需要操作 n 次把所有海鸥和食物都删除完，或判定无解。需要输出方案。

多测，$\sum n \leq 5000$。时限 $1s$。

hint

假如不需要考虑操作的顺序，也就是直接找到 n 对合法的海鸥和食物一起删除，应该怎么做。

qoj14742 海鸥奇遇

- 假如不需要考虑操作的顺序，那就是一个匹配问题。每只海鸥找到四个方向上第一份食物，如果中间的格子全是海鸥就在这只海鸥和这份食物之间连一条边。然后只需要做一次二分图匹配就可以了，如果原题有解，那显然一定可以找到一个这样的匹配。
- 找到这样的匹配之后，还需要考虑操作顺序的问题，每只海鸥与其匹配的食物中间的所有海鸥都应该在这只海鸥删除完之后才能删除，否则这只海鸥删除时它与食物中间就会有空格子。我们给有限制的海鸥之间连上有向边，就可以得到一个有向图，如果这个有向图没有环，那直接拓扑排序就可以得到合法的操作顺序。
- 而如果有环的话，我们可以通过不断调整使得最终没有环。
- 我们先设一个势能 ϕ 记为所有海鸥和匹配的食物之间的距离的和。首先初始的势能 ϕ 是 $O(n)$ 的，由于每只海鸥与其匹配的食物之间的所有格子都是海鸥，因此距离和可以被转换成每只海鸥被别的匹配“跨越”的次数的和，而一只海鸥只可能被四个方向上的第一份食物所在的匹配“跨越”，因此总势能 ϕ 是 $O(n)$ 的。

qoj14742 海鸥奇遇

- 考虑怎么调整，我们随便找到有向图上的一个环，记环上的点依次为 $a_0, a_1, \ldots, a_{t-1}$，其中的顺序是每个 a_i 应该在 a_{i+1} 之前删除。
- 我们再记 a_i 这只海鸥匹配的食物为 b_i，我们直接将匹配调整为对于所有 $i \in [0, t-1]$，$a_{(i+1)\%t}$ 与 b_i 匹配。
- 由于 $a_{(i+1)\%t}$ 一定在 a_i 和 b_i 的路径上，此时的匹配依旧是合法的，且总势能 ϕ 一定减小了。
- 而任意局面一定有 $\phi \geq 0$，所以在有限次调整后有向图一定就没有环了，而初始 ϕ 是 $O(n)$ 的，所以调整次数也是 $O(n)$ 的，而单次调整的时间复杂度刚好是 $O(n+\phi)$ 的，所以总时间复杂度 $O(n^2)$。
- 由这个势能还能导出一个做法，就是直接将海鸥与食物匹配的边权设置成两者的距离，然后直接求二分图最小权完美匹配，此时建出来的有向图就一定是无环的，时间复杂度也是 $O(n^2)$，可惜这个做法常数太大难以通过。

qoj18339 One, Two, Three

statement

Source: ptz winter 2026 day1 C

给定一个长度为 n 的序列 a_i，其中 $a_i \in \{1,2,3\}$。同时给一个长度为 n 的序列 b_i，代表第 i 个元素的权值。你可以进行任意次以下操作：选择该序列的一个区间，满足该区间的 a_i 的和为 4 或 8，然后删除这个区间，删除之后 a 和 b 序列的左右两侧会重新拼起来形成一个新序列。你需要最小化最后的序列的 a_i 的和，如果有多种方案能达到最小值，你需要最大化最后的序列的 b_i 的和。需要构造方案。

多测，$n, \sum n \le 3 \times 10^5, a_i \in \{1,2,3\}, 1 \le b_i \le 99$。时限 $1s$。

qoj18339 One, Two, Three

statement

Source: ptz winter 2026 day1 C

给定一个长度为 n 的序列 a_i，其中 $a_i \in \{1,2,3\}$。同时给一个长度为 n 的序列 b_i，代表第 i 个元素的权值。你可以进行任意次以下操作：选择该序列的一个区间，满足该区间的 a_i 的和为 4 或 8，然后删除这个区间，删除之后 a 和 b 序列的左右两侧会重新拼起来形成一个新序列。你需要最小化最后的序列的 a_i 的和，如果有多种方案能达到最小值，你需要最大化最后的序列的 b_i 的和。需要构造方案。

多测，$n, \sum n \le 3 \times 10^5, a_i \in \{1,2,3\}, 1 \le b_i \le 99$。时限 $1s$。

hint

考虑什么样的序列可以删空。

qoj18339 One, Two, Three

- 这个题的重点在于什么样的 a_i 序列是可以直接删空的。
- 首先一个明显的必要条件是 a_i 的和是 4 的倍数。
- 再手玩一下可以发现没有 3 的时候一定可以删空，也就是说 3 是特殊的，应该先把 3 删完，又发现 332 和 31 是删掉 3 最“高效”的方法。因此结论是记 $1,2,3$ 的个数分别为 s_1,s_2,s_3，能删完当且仅当 $s_1+2s_2\geq s_3$。
- 考虑如何证明这个结论。必要性考虑一个 1 最多只能删掉一个 3，一个 2 最多只能删掉两个 3，也就是上文说的最“高效”，因此这个显然是必要的。
- 充分性考虑在满足条件的情况下直接构造。
- 首先不停地找 31 和 332 删掉，直到找不到为止，此时一定还满足条件。

qoj18339 One, Two, Three

- 由于目前已经没有 31 和 332 了，那也就一定不会有两个以上连续的 3，且与 3 相邻的一定是 2 而不是 1，此时我们直接从左往右扫描整个序列，维护一个栈，每次往栈里加入元素，如果栈顶的若干个元素可以删除就直接删除，由于鸽巢原理，栈内元素到达 4 时一定会出现一个区间的和是 4 的倍数，而长度在 4 以内不能删的就只有 3333，由于前面的性质，在栈内是两个 3 时下一个一定加入一个 2，从而触发 332 删除，因此栈内不可能出现 3333，所以这样扫描到最后一定能全部删除完。
- 知道什么样的序列可以删除完之后就简单了。直接 dp_i 表示前缀 i 的答案，转移使用树状数组即可，不过多赘述。
- 时间复杂度 $O(n \log n)$。

Outline

1. Group 1
   1.1. qoj22881 Subset Sums
   1.2. qoj18316 插松枝
   1.3. qoj18428 Sort
   1.4. qoj14742 海鸥奇遇
   1.5. qoj18339 One, Two, Three
2. Group 2
   2.1. qoj18302 Remix
   2.2. qoj23137 Graph Built by XOR
   2.3. qoj23130 Orientation of a Graph
   2.4. qoj22886 Merge Sequence
   2.5. qoj7782 Ursa Minor

qoj18302 Remix

statement

Source: ptz winter 2026 day1 C I

给一个由 n 个整数组成的可重集 S。你需要重复执行以下三步若干次，直到 S 中只剩下一个整数：

1. 选择 S 的一个可重子集 T，满足 $|T| \geq 2$。
2. 从 S 中删除 T 中的元素。
3. 将 $\max(T) - \min(T)$ 这个数插入到 S 中。

求 S 中最后剩下的那个数最大是多少，要构造方案。

$n \leq 10^5, 0 \leq S_i \leq 10^5$。时限 $1s$。

qoj18302 Remix

statement

Source: ptz winter 2026 day1 C I

给一个由 n 个整数组成的可重集 S。你需要重复执行以下三步若干次，直到 S 中只剩下一个整数：

1. 选择 S 的一个可重子集 T，满足 $|T| \geq 2$。
2. 从 S 中删除 T 中的元素。
3. 将 $\max(T) - \min(T)$ 这个数插入到 S 中。

求 S 中最后剩下的那个数最大是多少，要构造方案。

$n \leq 10^5, 0 \leq S_i \leq 10^5$。时限 $1s$。

hint

假如你可以选 S 的一个子集进行操作，并且你要让最后操作出来的数最小（而不是最大），要怎么做。

qoj18302 Remix

- 首先有个结论是可以用 S 中除了最大值以外的数操作出一个最小值（可以不用完），再用最大值减去这个操作出来的最小值就是答案。证明也简单，记最后一次操作的集合为 T，若 $\max\{T\} \neq \max\{S\}$，则将操作中的 $\max\{S\}$ 与 $\max\{T\}$ 交换一下，交换完之后最后一次操作的集合记为 T'，那么有 $|\min\{T'\} - \min\{T\}| \leq \max\{S\} - \max\{T\}$，而 $\max\{T'\} = \max\{S\}$，所以 $\max\{T'\} - \min\{T'\} \geq \max\{T\} - \min\{T\}$，也就是说这样调整之后答案不劣，所以一定存在一个最优解是让最大值留到最后再操作的。
- 有这个结论之后我们就直接考虑怎么操作出一个最小的数。
- 容易发现所有数对最后答案的贡献系数一定是 $0, 1, -1$，因此直接猜测 $\sum_{i=1}^{n} k_i a_i (k_i \in \{0, 1, -1\})$ 能取到的非负最小值就是答案。
- 证明直接构造，由于取到的是非负最小值，因此要么此时只剩一个元素了，要么一定能找到 $k_i = -1, k_j = 1$，如果 $a_i < a_j$，那就删掉 a_i, a_j 然后加入 $a_j - a_i$，这个新加入的数的系数 $k = 1$，如果 $a_i > a_j$，那就删掉 a_i, a_j 然后加入 $a_i - a_j$，这个新加入的数的系数 $k = -1$，一直这样重复操作就能构造出答案。

qoj18302 Remix

- 现在的问题变成找到 k_i 数组使得 $\sum_{i=1}^{n} k_i a_i$ 取到非负最小值。
- 容易发现当 n 大的时候答案总是 0。
- 于是可以想到一个暴力做法，枚举所有 $k_i \in \{0,1\}$ 一共 2^n 组系数，然后选择任意两组系数 $k_{1,i}, k_{2,i}$，将其相减可得到 $k_{3,i} = k_{1,i} - k_{2,i}$，显然 $k_{3,i} \in \{-1,0,1\}$，而且所有 $k_i \in \{-1,0,1\}$ 一共 3^n 组系数都可以由至少一对 $k_i \in \{0,1\}$ 的系数相减得到，所以直接找到 2^n 组系数求和之后排序，将相邻的两组系数相减取最小值就是答案。
- 这样直接做是 $O(2^n)$，我们发现 $2^n > nV$ 时，2^n 组系数一定会出现相同的，此时答案就已经是 0 了，因此当 $n > \log V + 2\log\log V$ 时，随便取 $\log V + 2\log\log V$ 个数跑就行了。
- 时间复杂度 $O(V \log^2 V)$。

qoj23137 Graph Built by XOR

statement

Source: 5th Ucup extra stage 1 L

给定一个长度为 n 的数组 a_i。由此数组构建一个包含 n 个点的简单无向带权图：点 i 和点 j 之间有边当且仅当 $i \neq j, a_i \oplus a_j \leq m$（$\oplus$ 是异或），这条边的边权为 $a_i + a_j$。

求顶点 1 到顶点 n 的最短路长度，如果不存在，则输出 -1。

$n \leq 2 \times 10^5, 0 \leq m \leq 2^{30}, 0 \leq a_i \leq 2^{30}$。时限 $1s$。

qoj23137 Graph Built by XOR

- 首先对 a_i 建 trie 树，对每个 i，与 i 有连边的点可以分成 trie 树上的 $\log V$ 个子树。
- 然后直接跑类似 dij 的最短路。每次从优先队列中找到 $dis_i + a_i$ 最小的点，然后暴力找到与 i 有连边的 $\log V$ 的子树上没被访问过的点，那么这些点的距离就是 $dis_i + a_i + a_j$（因为 i 是当前 $dis_i + a_i$ 最小的点），接着将这些点的 $dis_j + a_j$ 加入优先队列。这样每个 trie 树上的点只会被访问一次，每个点只会加入一次优先队列。
- 时间复杂度 $O(n \log V)$。

qoj23130 Orientation of a Graph

statement

Source: 5th Ucup extra stage 1 E

给定一张 n 个顶点的完全无向图，和一个正整数 d，你需要为每条边定向，使得对每个顶点都满足它的出度减去入度能被 d 整除。

输出任意一种合法定向方案，或者判定无解。

多测，$n \leq 1000, 2 \leq d \leq n, \sum n \leq 20000$。时限 $1s$。

qoj23130 Orientation of a Graph

statement

Source: 5th Ucup extra stage 1 E

给定一张 n 个顶点的完全无向图，和一个正整数 d，你需要为每条边定向，使得对每个顶点都满足它的出度减去入度能被 d 整除。

输出任意一种合法定向方案，或者判定无解。

多测，$n \leq 1000, 2 \leq d \leq n, \sum n \leq 20000$。时限 $1s$。

hint

$n = 2d$ 时怎么构造。

qoj23130 Orientation of a Graph

- 首先 n 是奇数的时候直接让每个点的入度等于出度即可，有很多种构造方法，这里不再赘述，下面只考虑 n 是偶数并且 d 是奇数的情况。
- 手玩容易发现 $2d > n$ 时是无解的。因此直接想 $n = 2d$ 要怎么构造，一个简单的构造是将 n 均分成两组，两组之间的边全部由第一组的点指向第二组的点，然后两组内部按照奇数的方法构造（因为 $\frac{n}{2} = d$ 是奇数）。
- 接下来想 $n > 2d$ 要怎么做。容易发现一个让 d 不变，n 增加 2 的增量构造，记当前点数为 m，则加入 $m+1, m+2$ 这两个点时，令 $k = \frac{m+d-1}{2}$，让 $m+1$ 指向 1 到 k，1 到 k 指向 $m+2$，$m+2$ 指向 $k+1$ 到 m，$k+1$ 到 m 指向 $m+1$，最后让 $m+1$ 指向 $m+2$，计算可得新的图仍然满足条件。
- 这样就做完了 $n \geq 2d$ 的所有情况了，直接构造 $n = 2d$ 再一步步增量构造即可。
- $n < 2d$ 时无解的证明也很简单，此时如果有解只能是一半的点出度减入度等于 d（记为左半边），另一半的点入度减出度等于 d（记为右半边）。

qoj23130 Orientation of a Graph

- 记左半边点的出度为 d_{out}，入度为 d_{in}，由 $d_{in}+d_{out}=n-1, d_{out}-d_{in}=d>\frac{n}{2}$ 可得 $d_{out}>\frac{3n-2}{4}, d_{in}<\frac{n-2}{4}$，而左半边的每个点最多只能向右半边连 $\frac{n}{2}$ 条边，因此每个点还必须向左半边连至少 $d_{out}-\frac{n}{2}>\frac{n-2}{4}$ 条边，也就是说左半边的点一共向左半边的点连大于 $\frac{n(n-2)}{8}$ 条边，而左半边的入度和小于 $\frac{n(n-2)}{8}$，所以必然是无解的。
- 时间复杂度 $O(n^2)$。

qoj22886 Merge Sequence

statement

Source: 4th Ucup semifinal J

给定 m 个序列，这些序列的总长度为 n，且 $[1,n]$ 中的每个整数在所有序列中都恰好出现一次。你需要将这 m 个序列归并成一个长度为 n 的排列 p，并让 $\sum_{i=1}^{n}[p_i=i]$ 最小，即让 $p_i=i$ 的位置最少。不需要构造方案。

多测，$\sum n\le 10^6,\sum m\le 10^6$。时限 $2s$。

qoj22886 Merge Sequence

statement

Source: 4th Ucup semifinal J

给定 m 个序列，这些序列的总长度为 n，且 $[1,n]$ 中的每个整数在所有序列中都恰好出现一次。你需要将这 m 个序列归并成一个长度为 n 的排列 p，并让 $\sum_{i=1}^{n}[p_i=i]$ 最小，即让 $p_i=i$ 的位置最少。不需要构造方案。

多测，$\sum n\le 10^6,\sum m\le 10^6$。时限 $2s$。

hint

序列长度都不大于 $\frac{n}{2}$ 时答案是多少。

qoj22886 Merge Sequence

- 首先记每个序列的长度为 k_i。
- 有个结论是当所有 $k_i \le \frac{n}{2}$ 时答案是 0。证明不难，设 fr_i 表示 i 这个数出现在第 fr_i 个序列中，一个归并方案中如果 $\forall i, fr_i \neq fr_{p_i}$，那么自然有 $\forall i, p_i \neq i$，即答案为 0，这样一个方案相当于第 i 个序列中的数只能放到 $fr_j \neq i$ 的位置，由于所有 $k_i \le \frac{n}{2}$，因此根据 Hall 定理，一定存在一个这样的方案。
- 现在考虑存在一个序列 t，使得 $k_t > \frac{n}{2}$ 的情况。
- 这里还有一个结论，就是只用考虑第 t 个序列中的数造成的贡献，即只用考虑 $fr_i = t, p_i = i$ 这些位置造成的贡献。
- 证明有点复杂，就是考虑不停地调整方案使得在第 t 个序列中的数造成的贡献不变大的同时让 $fr_i \neq t, fr_{p_i} \neq t$ 的位置越来越少。

qoj22886 Merge Sequence

- 具体的，每次找到一个 $fr_i \neq t, fr_{p_i} \neq t$ 的位置 i，然后找到 i 的前面/后面第一个 $fr_j = t, fr_{p_j} = t$ 的位置 j，然后交换 fr_{p_i} 和 fr_{p_j} 再调整对应的 p，这样会让 $fr_i \neq t, fr_{p_i} \neq t$ 的位置少一个，且由于 i 和 j 之间 $fr_k = t$ 的位置都满足 $fr_{p_k} \neq t$（因为 j 是第一个满足 $fr_j = t, fr_{p_j} = t$ 的），所以第 t 个序列中的数造成的贡献也不会变大。
- 这样调整到最后不在第 t 个序列中的数就没有贡献了。
- 有这个结论之后就可以 dp 了，记 a_i 表示第 t 个序列中的第 i 个数，设 $dp_{i,j}$ 表示在前 $i+j$ 个位置放了 i 个在第 t 个序列中的数和 j 个不在第 t 个序列中的数时最小贡献是多少。转移很显然，如下：

$$dp_{i,j} = \min\{dp_{i,j-1}, dp_{i-1,j} + [a_i = i + j]\}$$

qoj22886 Merge Sequence

- 这个 dp 由转移式可以得出第二维是单调的，即 $dp_{i,j} \le dp_{i,j-1}$。再观察一下可以发现 dp_i 和 dp_{i-1} 不同的地方很少，只有在第二维 $j = a_i - i$ 的位置有可能不同。再简单分讨即可得出当 $dp_{i-1,a_i-i} = dp_{i-1,a_i-i-1}$ 时，dp_i 与 dp_{i-1} 两个数组完全相同，否则 $dp_{i,a_i-i} = dp_{i-1,a_i-i} + 1$，其他位置依然相同。这样就可以快速求出这个 dp 了，从小到大枚举 i 的同时维护一个 dp 第二维的数组即可。
- 时间复杂度 $O(n)$。

qoj7782 Ursa Minor

statement

Source: ICPC Shenyang 2023 F

一个关卡由一个长度为 n 的目标序列 a_i 和长度为 m 的工具序列 b_i 组成。初始你有一个长度为 n 且值全为 0 的环形数组，你每次操作可以从工具序列中选择一个 b_i 并选择一个任意实数 x，然后选择环形数组上一个长度为 b_i 的区间，让这个区间上的数都加上 x。如果最后你能让这个环形数组从任意起点、任意方向开始与目标序列 a_i 完全相同，那么这个关卡就通关了。

给定一个长度为 N 的序列 A_i，一个长度为 M 的序列 B_i，有 Q 次操作，分为两种：

1. 给定 p, v，修改序列 $A_p \leftarrow v$。
2. 给定 l, r, s, t，询问以 $A_l, \ldots, A_r$ 为目标序列，以 $B_s, \ldots, B_t$ 为工具序列的关卡能否通关，这里保证 $\forall i \in [s, t], B_i \leq r - l + 1$。

$N, M, Q \leq 2 \times 10^5, 0 \leq A_i \leq 10^9, 1 \leq B_i \leq N$。时限 $3s$。

qoj7782 Ursa Minor

statement

Source: ICPC Shenyang 2023 F

一个关卡由一个长度为 n 的目标序列 a_i 和长度为 m 的工具序列 b_i 组成。初始你有一个长度为 n 且值全为 0 的环形数组，你每次操作可以从工具序列中选择一个 b_i 并选择一个任意实数 x，然后选择环形数组上一个长度为 b_i 的区间，让这个区间上的数都加上 x。如果最后你能让这个环形数组从任意起点、任意方向开始与目标序列 a_i 完全相同，那么这个关卡就通关了。

给定一个长度为 N 的序列 A_i，一个长度为 M 的序列 B_i，有 Q 次操作，分为两种：

1. 给定 p, v，修改序列 $A_p \leftarrow v$。
2. 给定 l, r, s, t，询问以 $A_l, \ldots, A_r$ 为目标序列，以 $B_s, \ldots, B_t$ 为工具序列的关卡能否通关，这里保证 $\forall i \in [s, t], B_i \leq r - l + 1$。

$N, M, Q \leq 2 \times 10^5, 0 \leq A_i \leq 10^9, 1 \leq B_i \leq N$。时限 $3s$。

hint

若每次询问都有 $s = t$ 那要怎么做。

qoj7782 Ursa Minor

- 先考虑单个关卡能不能通关。
- 由裴祖定理可以得到一个结论，通过不停地用不同的工具序列对不同区间 $\pm x$，可以做到对一个长度为 $d = \gcd\{n, b_1, b_2, \ldots, b_m\}$ 的区间 $\pm x$。
- 然后一个关卡能通关的充要条件是 $\sum_{i \bmod d=0} a_i = \sum_{i \bmod d=1} a_i = \cdots = \sum_{i \bmod d=d-1} a_i$。首先必要性显然，因为初始全 0 的数组满足这个条件，每次操作后数组也都满足这个条件，所以目标数组肯定要满足这个条件。充分性考虑直接构造方案，从 1 到 $n-d+1$ 枚举 i，通过操作 $[i, i+d-1]$ 这个区间使得 i 这个位置与目标数组相同且不影响前面的位置，到最后只剩下 $n-d+2$ 到 n 这 $d-1$ 个位置，由条件可得最后这些位置必定已经和目标数组相同。
- 回到原问题，首先用线段树可以简单求出 d，然后充要条件的判断可以用哈希。找一个 $base$ 和一个 P，求出 $(\sum_{i=l}^{r} A_i base^{i \bmod d}) \bmod P$，判断这个数是否与 $(\sum_{i=0}^{d-1} \frac{\sum_{j=l}^{r} A_j}{d} base^i) \bmod P$ 相同。
- 现在的问题只剩下如何求出这个哈希值了。

qoj7782 Ursa Minor

- 这个形式容易想到根号分治。
- 对于 $d \le \sqrt{n}$ 的询问，我们用 $\sqrt{n}$ 个数据结构维护 $d = 1, 2, \ldots, \sqrt{n}$ 下的 $A_i base^{i \bmod d}$，每次修改需要在 $\sqrt{n}$ 个数据结构上做单点修改，每次询问需要在一个数据结构上做区间求和，因此直接用 $O(1)$ 修改 $O(\sqrt{n})$ 查询的分块即可。
- 对于 $d > \sqrt{n}$ 的询问，我们用一个数据结构维护 $A_i base^i$，然后我们对最多 $\frac{n}{d}$ 个 $\lfloor \frac{i}{d} \rfloor$ 相同的区间分别求和之后偏移一下再加起来即可，每次修改需要一次单点修改，每次询问需要 $\sqrt{n}$ 次区间求和，因此用 $O(\sqrt{n})$ 修改 $O(1)$ 查询的分块即可。
- 时间复杂度 $O(n\sqrt{n})$。

Thank you for your listening!